Laboratório de Projeto de Software · PUC Minas

Sistema de Moeda Estudantil

Sprint 3 · Sistema completo

Reconhecer o mérito dos alunos com uma moeda virtual, trocada por vantagens em empresas parceiras.





Integrantes

Cleverson Resende

Rafael Lopes

Kaio Ferreira

Matheus Augusto

Problema e proposta

Como um programa de pontos: quem se destaca ganha moedas e troca por vantagens.



Professores

Reconhecem o mérito dos alunos



Alunos

Acumulam moedas virtuais



Empresas parceiras

Oferecem produtos e descontos

Sprint 3: ciclo completo — cadastros, login, distribuição, extrato, vantagens e resgate com cupom.

Documentos de modelagem

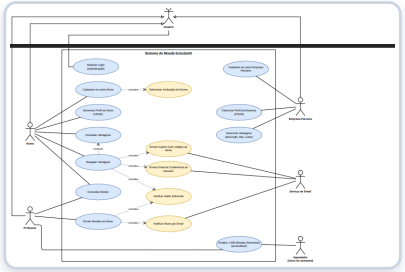
1



Histórias de Usuário

Necessidades

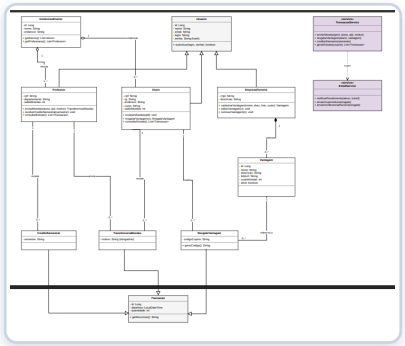
2



Casos de Uso

Funcionalidades

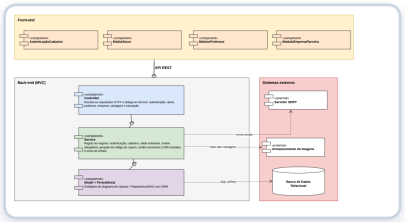
3



Classes

Entidades

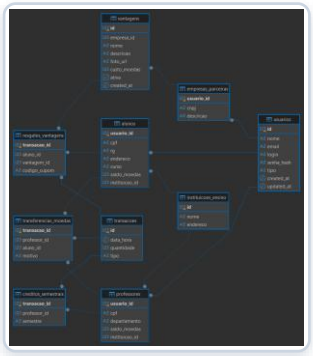
4



Componentes

Arquitetura

5

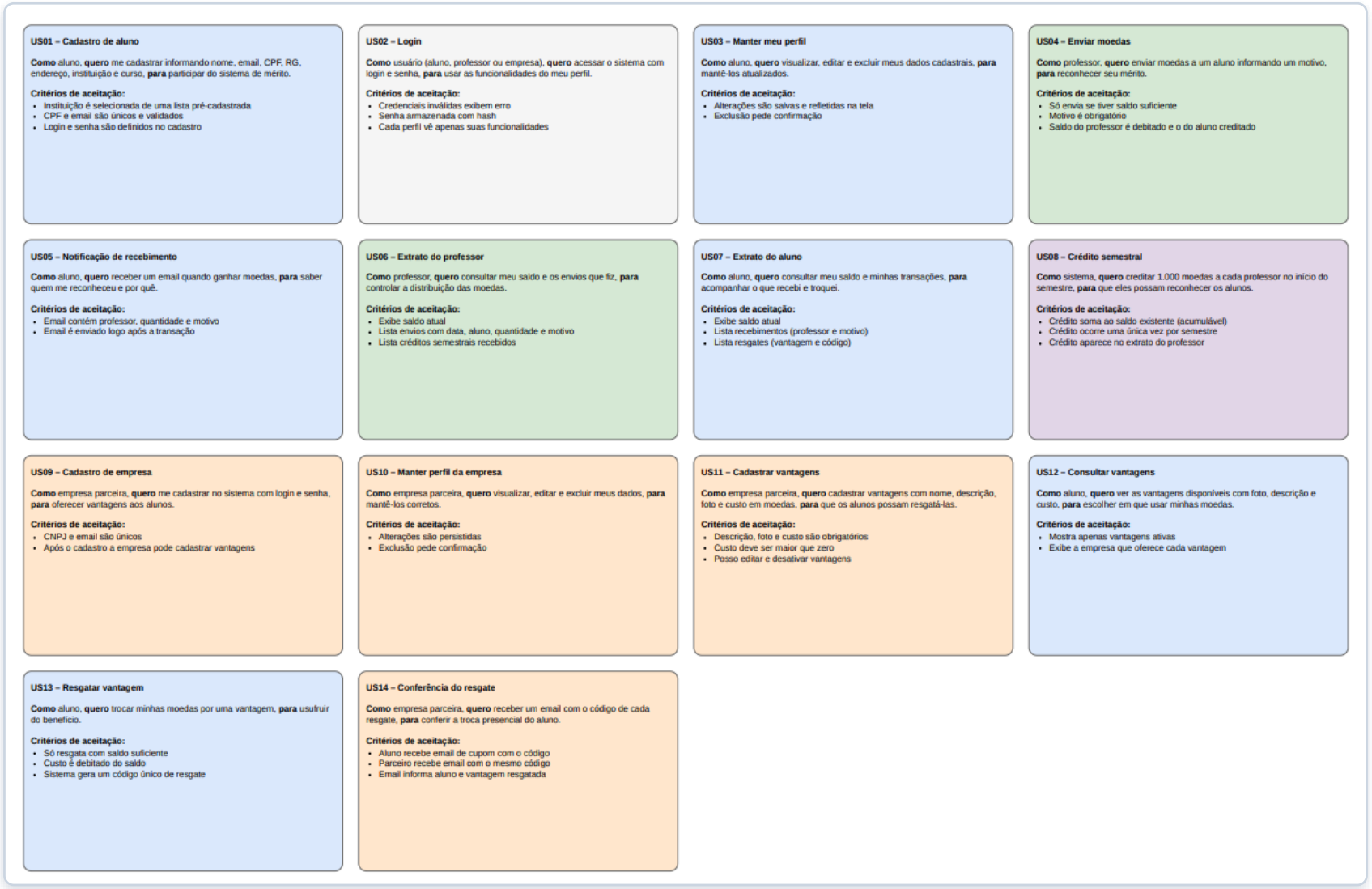


Modelo ER

Dados

Cada documento é uma visão diferente, até chegar à implementação.

Histórias de Usuário

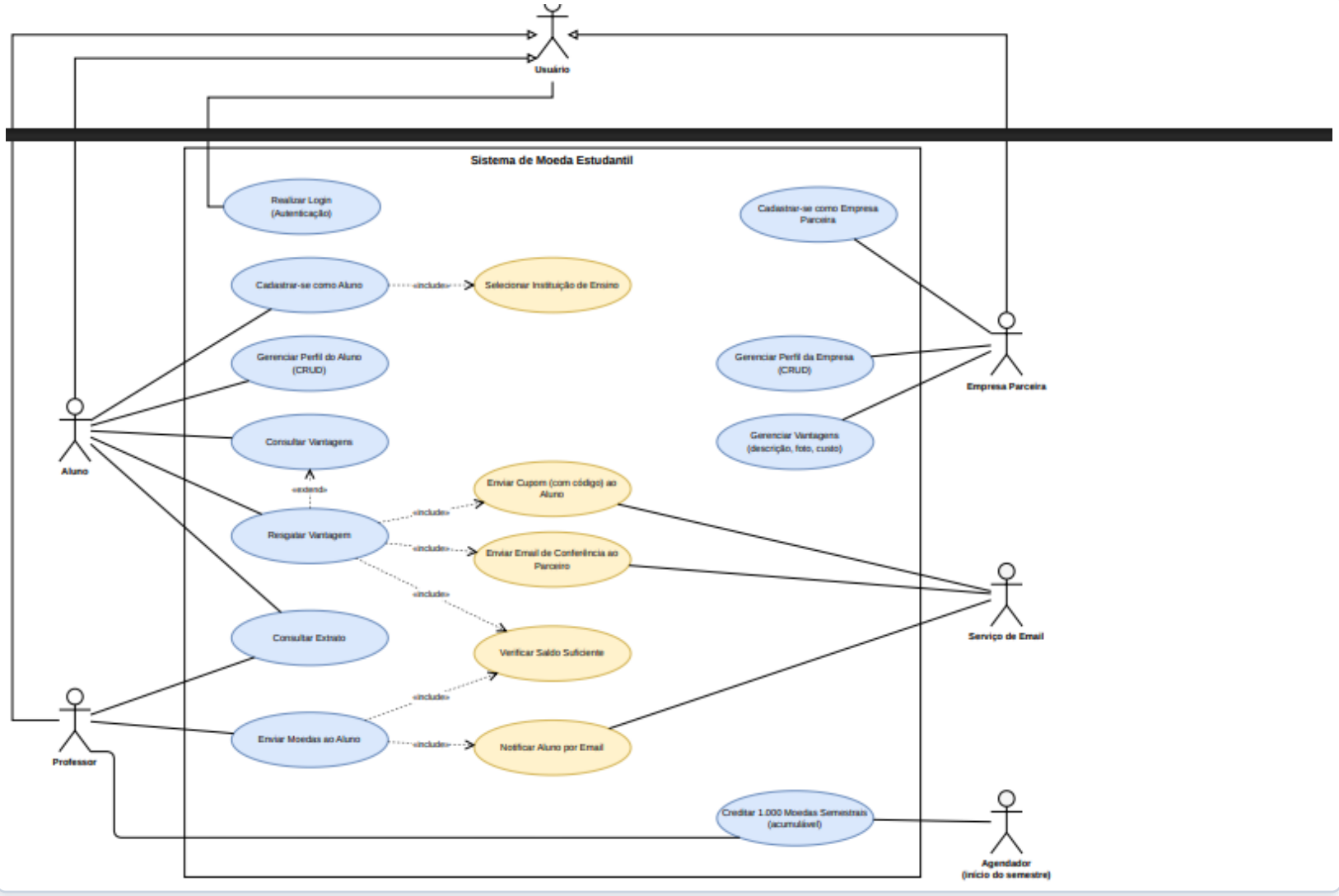


Etapa 1 de 5

Necessidades

O que cada usuário precisa: cadastro, login e uso da moeda.

Diagrama de Casos de Uso



Etapa 2 de 5

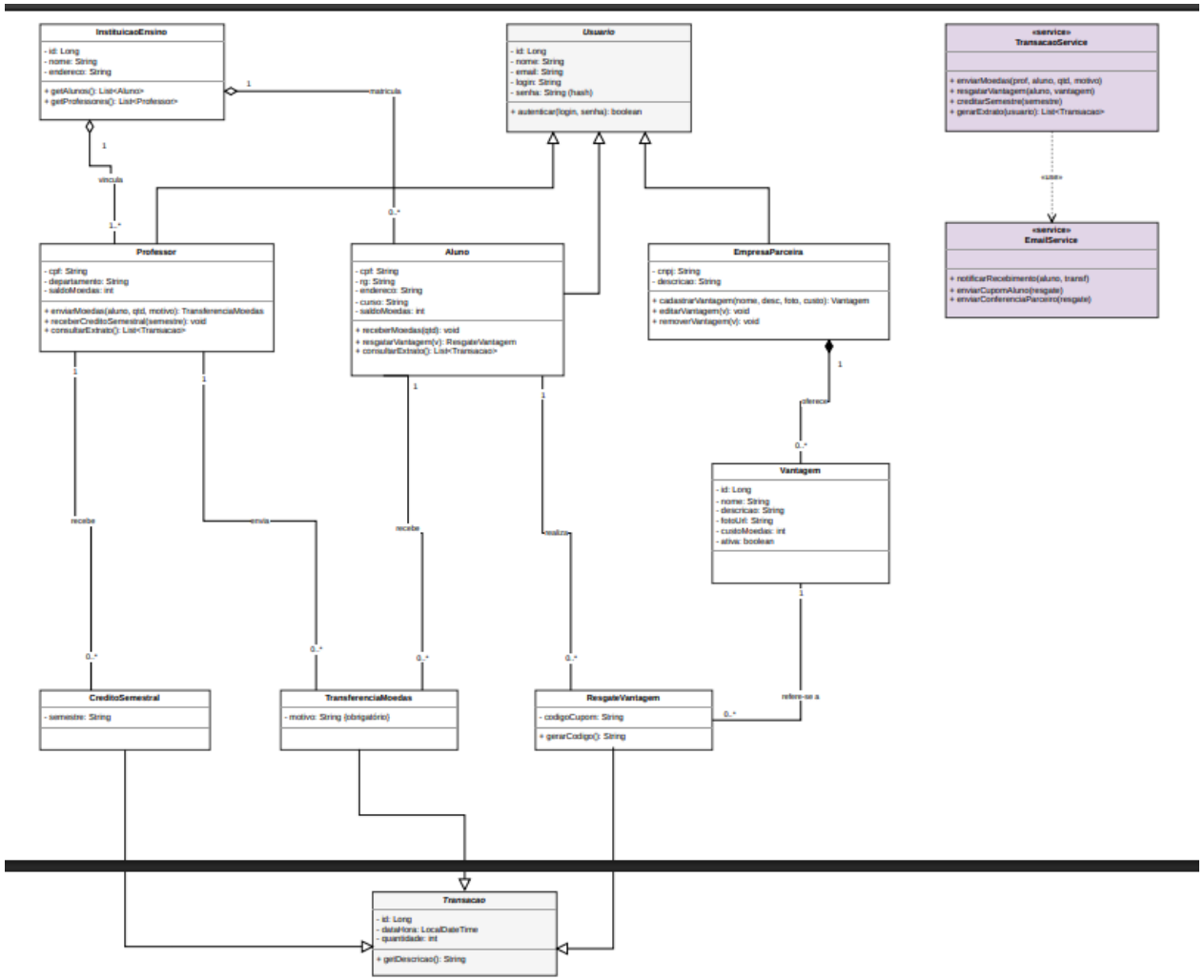
Funcionalidades

Atores: aluno, professor e empresa parceira.

Cadastro e login

Moedas: implementado

Diagrama de Classes



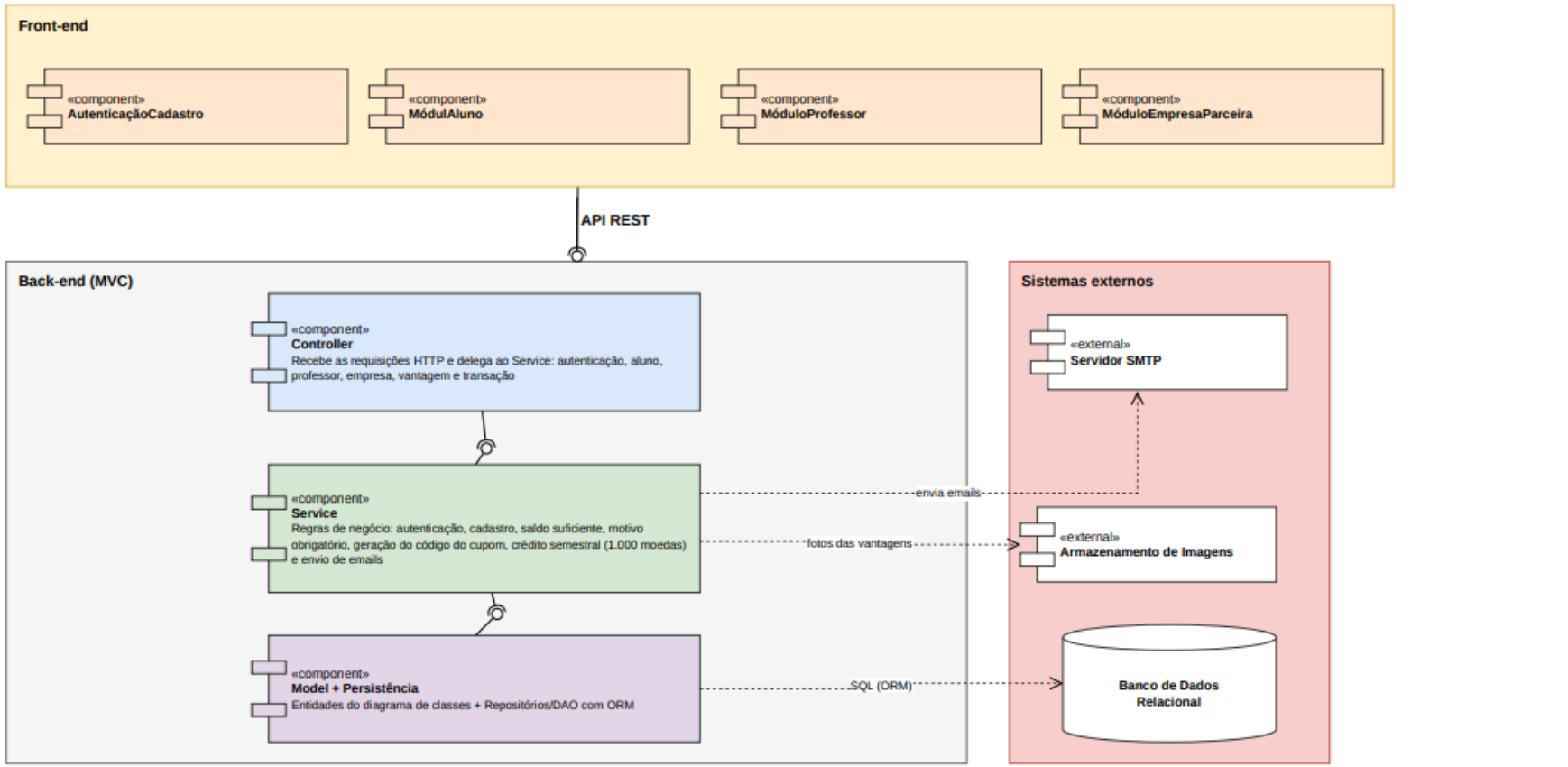
Etapa 3 de 5

Entidades

- **Usuario**
  - **Aluno**
  - **EmpresaParceira**
  - **Professor**

Base dos Models em src/models/

Diagrama de Componentes

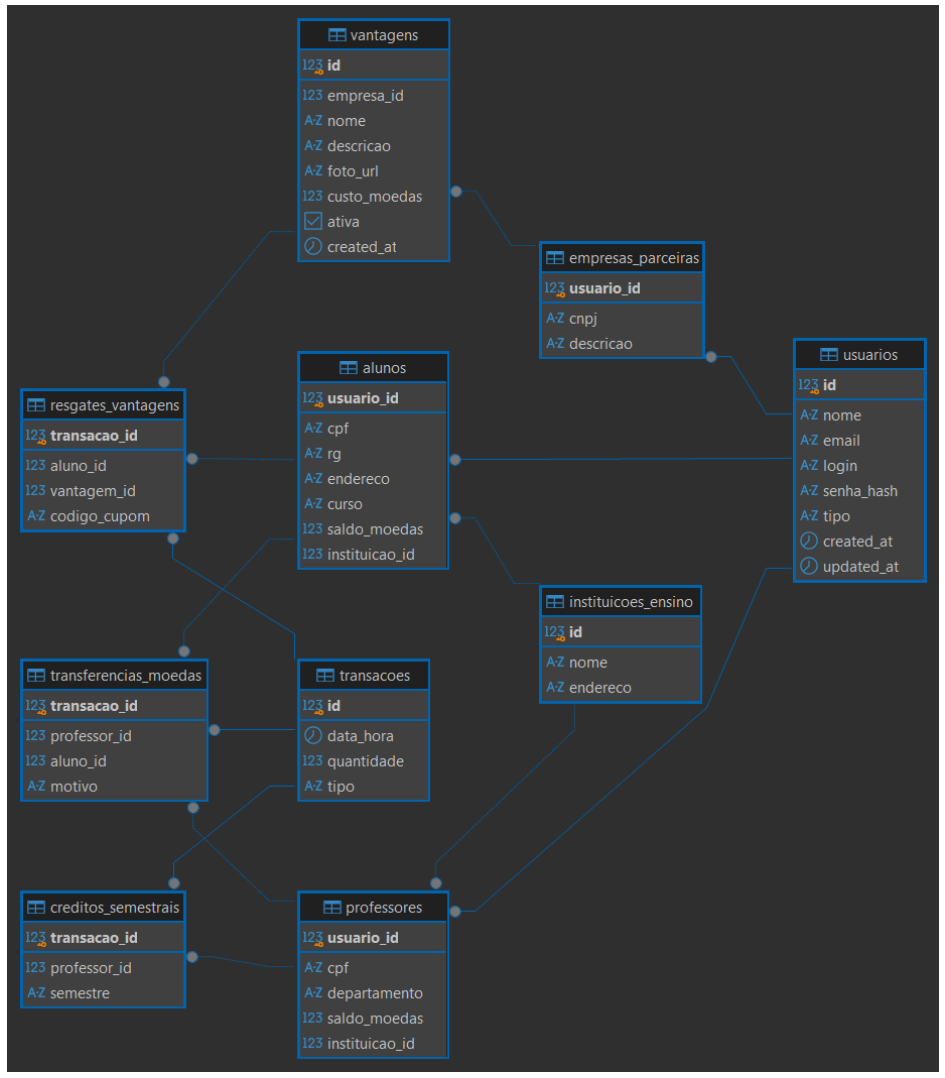


Etapa 4 de 5

Arquitetura

Como as partes técnicas se conectam.

Diagrama Entidade-Relacionamento



Etapa 5 de 5

Dados

- `usuarios`
- `alunos`
- `professores`
- `empresas_parceiras`
- `instituicoes_ensino`
- `vantagens`
- `transacoes`

usuarios guarda os dados comuns. Chaves estrangeiras e ON DELETE CASCADE mantêm a integridade.

Arquitetura geral

Frontend	Telas e requisições
API REST	Rotas HTTP em server.js
Controllers	Recebem e respondem
Services	Validam e aplicam regras
DAOs	Executam o SQL
Models	Entidades do sistema
SQLite	Armazena os dados



Como um prédio: cada andar tem uma função.

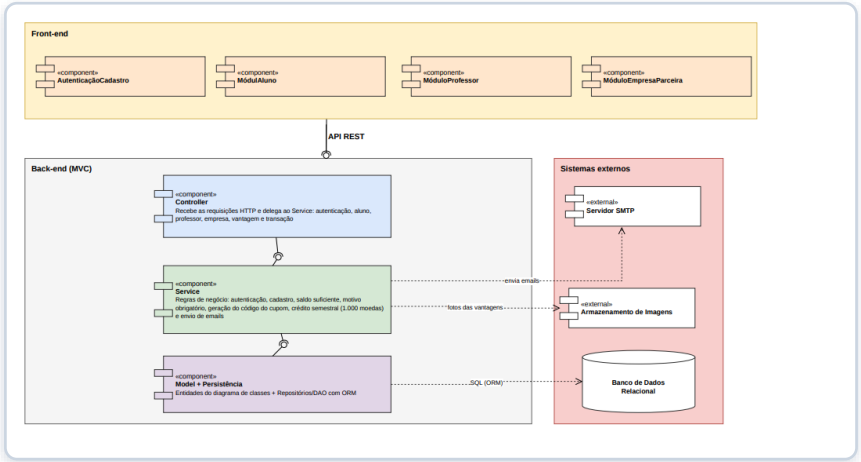
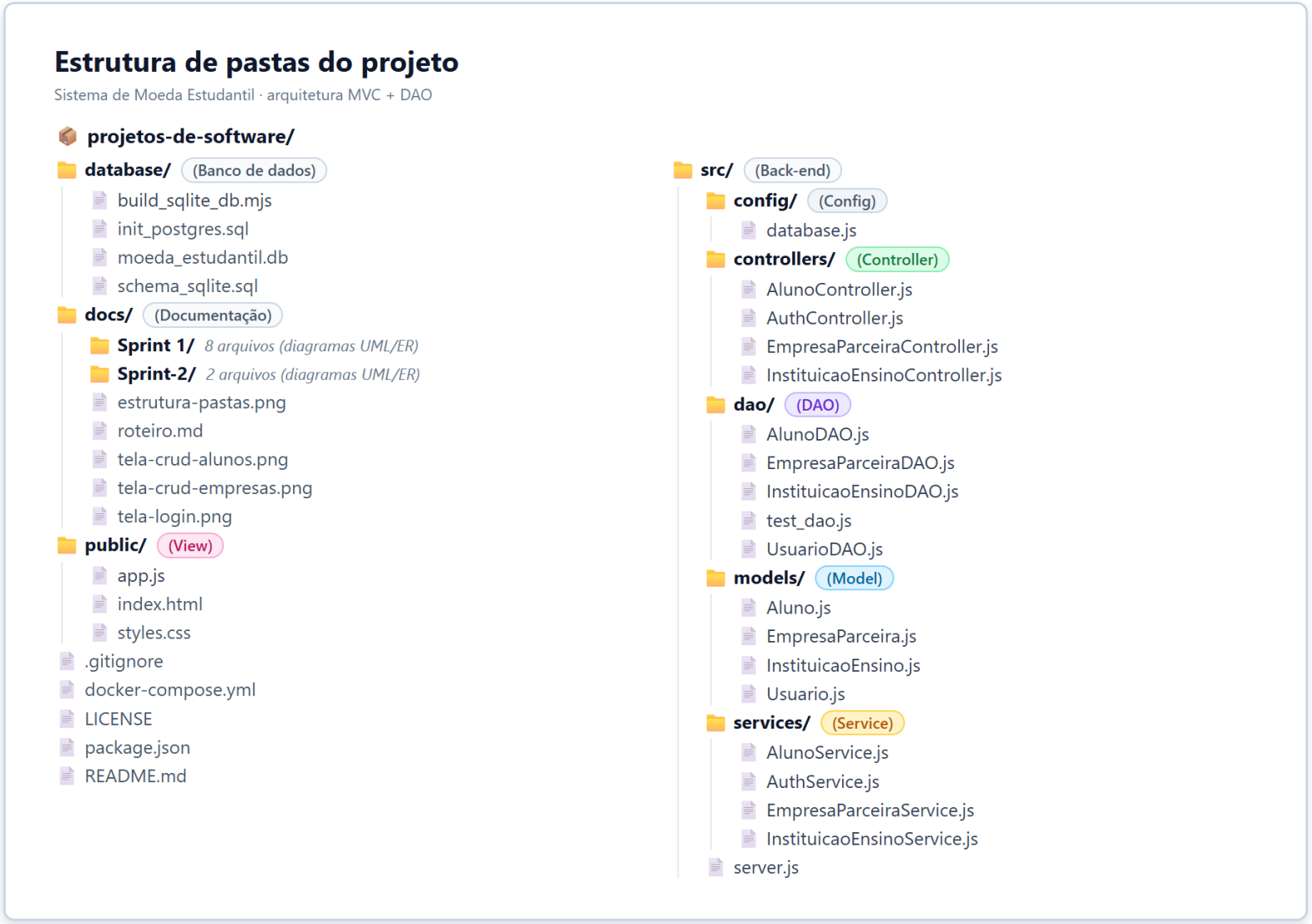


Diagrama de componentes (slide 7)

MVC e organização do código



Model

`src/models/`

View

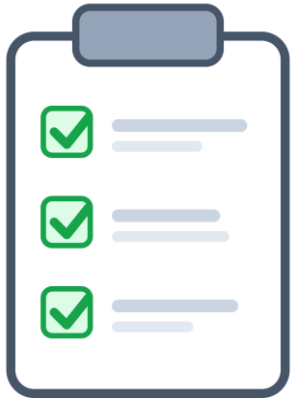
`public/`

Controller

`src/controllers/`

Interface, dados e controle separados.

Services e regras de negócio



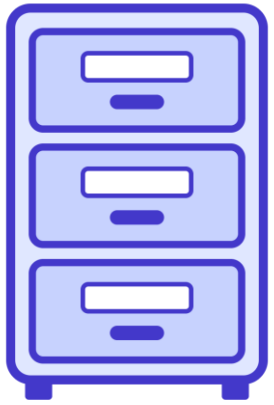
Como um guichê: confere antes de aceitar.

- ✓ Campos obrigatórios
- ✓ E-mail e login únicos
- ✓ RG, CPF e CNPJ válidos
- ✓ Instituição de ensino confirmada
- ✓ Login e senha conferidos

Regras antes de gravar:

Controller → **Service (valida)** → **DAO (grava)**

DAO e persistência



Como um arquivista: guarda e busca.

Operações de CRUD

`findAll` `findById` `create` `update` `delete`

`AlunoDAO · EmpresaParceiraDAO · UsuarioDAO`

O SQL fica só nos DAOs.

Service → DAO → SQLite

Herança, transações e segurança



Herança

- **Usuario**
  - **Aluno**
  - **EmpresaParceira**

Dados comuns + específicos



Transações

COMMIT: tudo certo

ROLLBACK: erro, desfaz

Tudo ou nada em usuarios e na tabela específica.

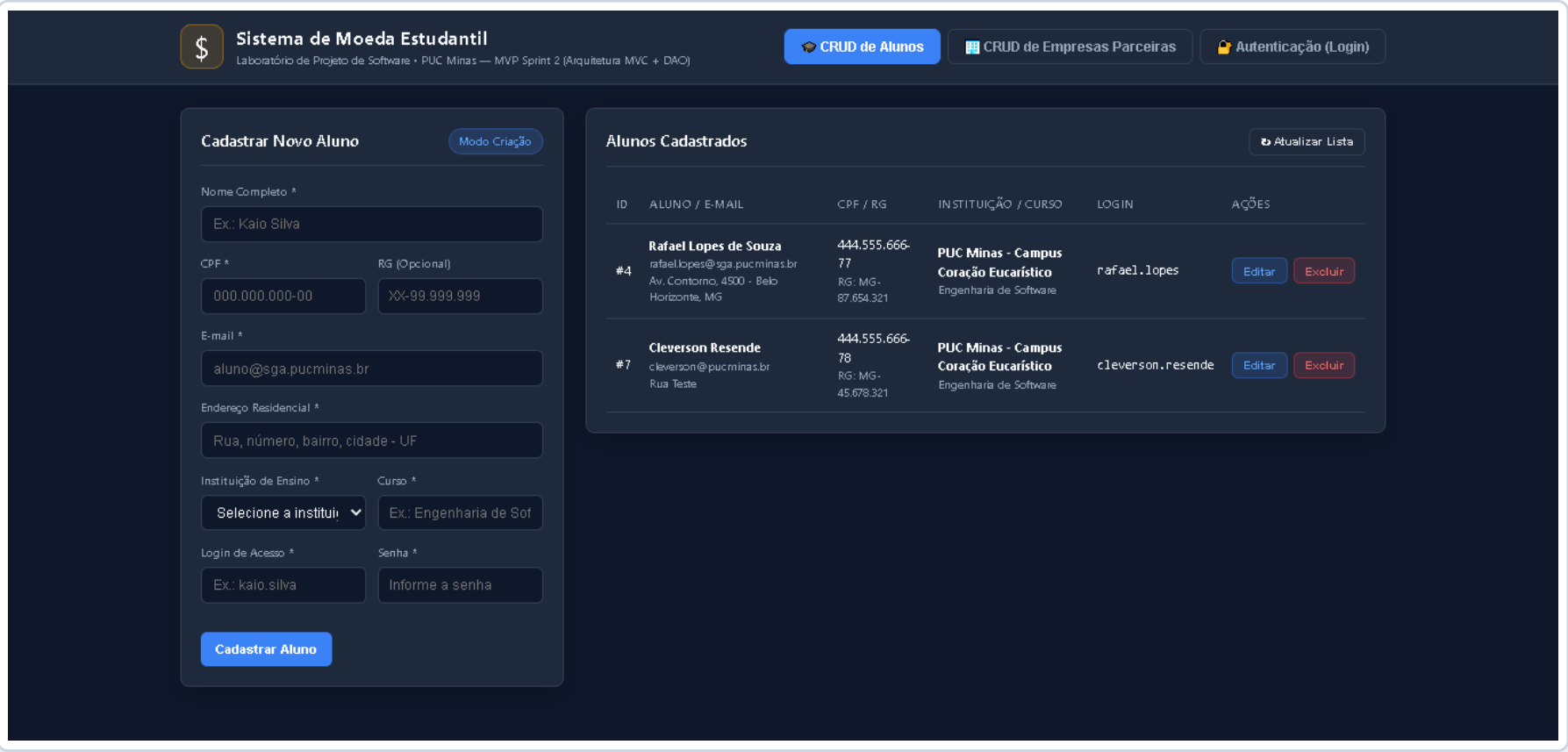


Segurança

- **PBKDF2**
- **SHA-256**
- **Salt aleatório**

Senha nunca em texto puro.

Tela de alunos

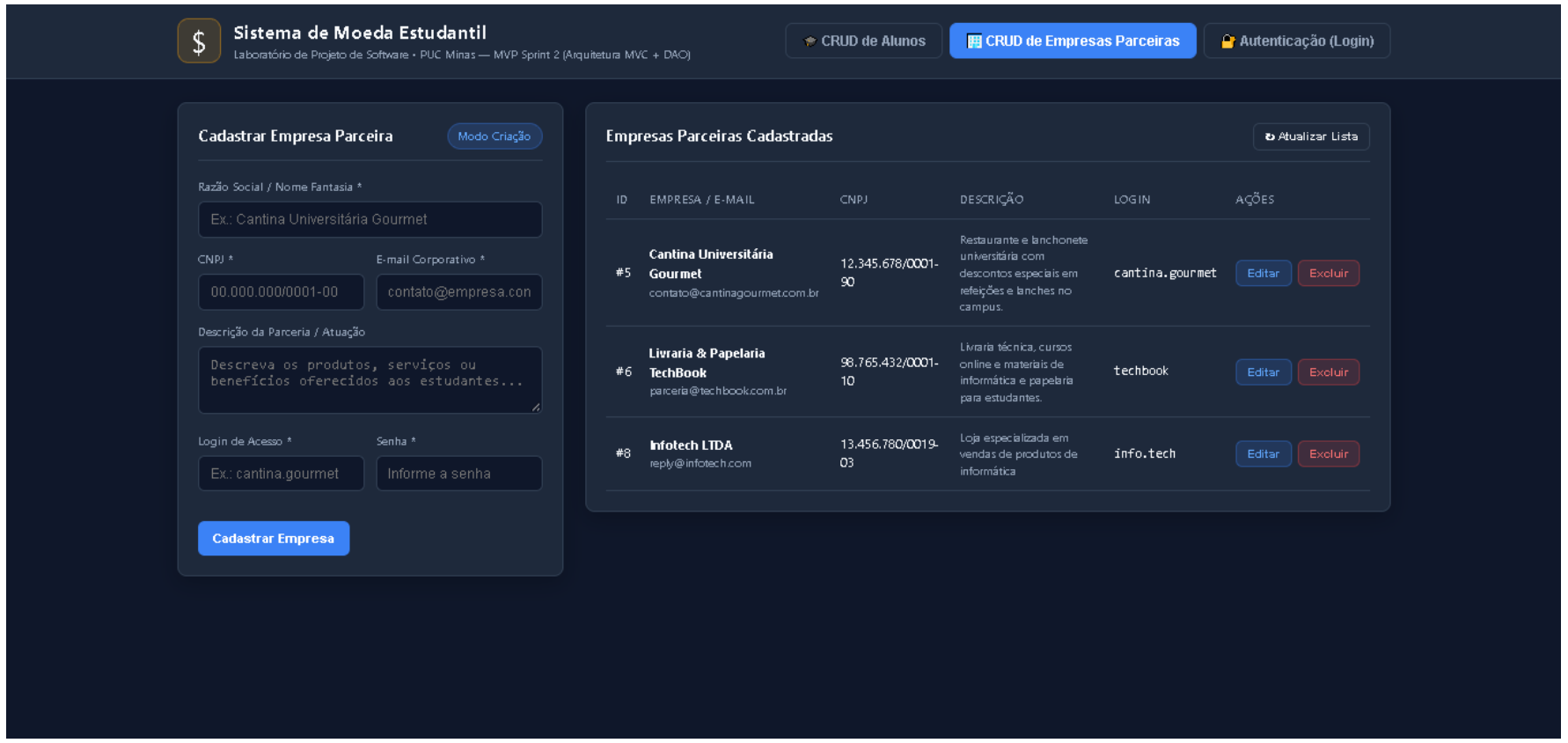




- Cadastrar
- Editar
- Excluir
- Listar
- Máscaras CPF/RG
- Instituição

Formulário → **POST /api/alunos** → **Service** → **DAO** → **SQLite**

Tela de empresas parceiras

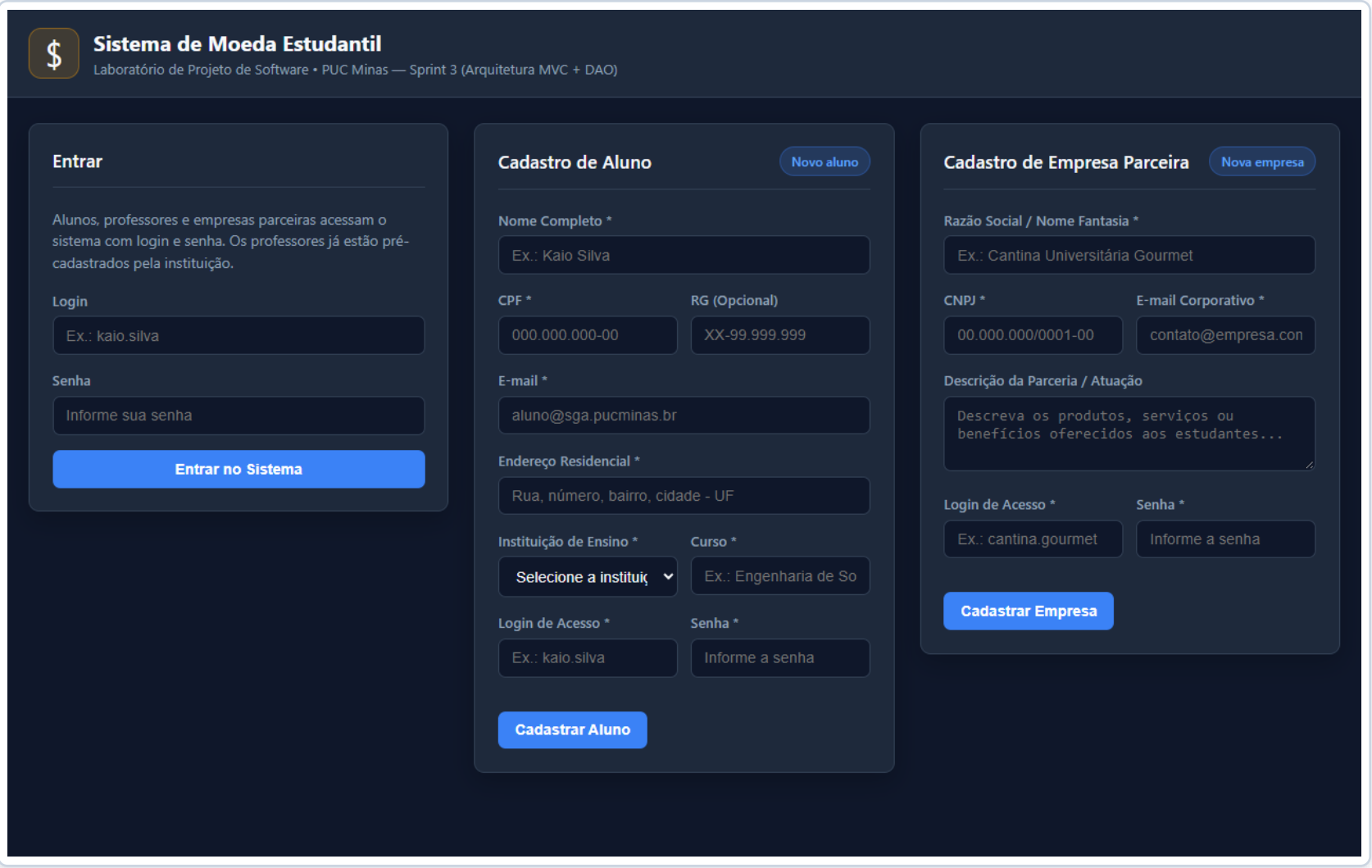




- Cadastrar
- Editar
- Excluir
- Listar
- CNPJ
- Descrição

Formulário → `POST /api/empresas` → Service → DAO → SQLite

Tela de login





Como uma portaria: confere o crachá.

AuthController

recebe login e senha

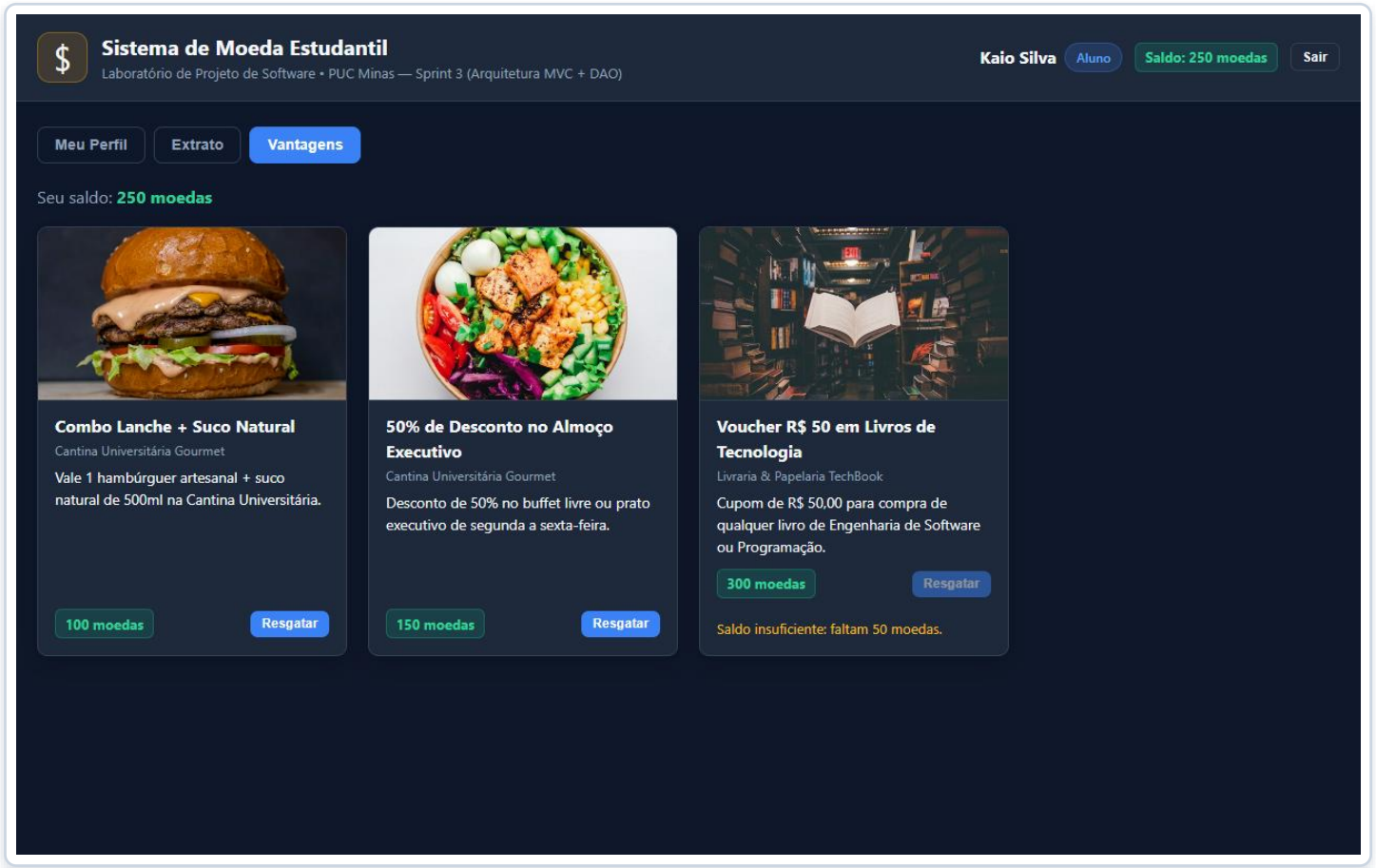
AuthService

compara com o hash

UsuarioDAO

busca no SQLite

Tela do aluno · Vantagens

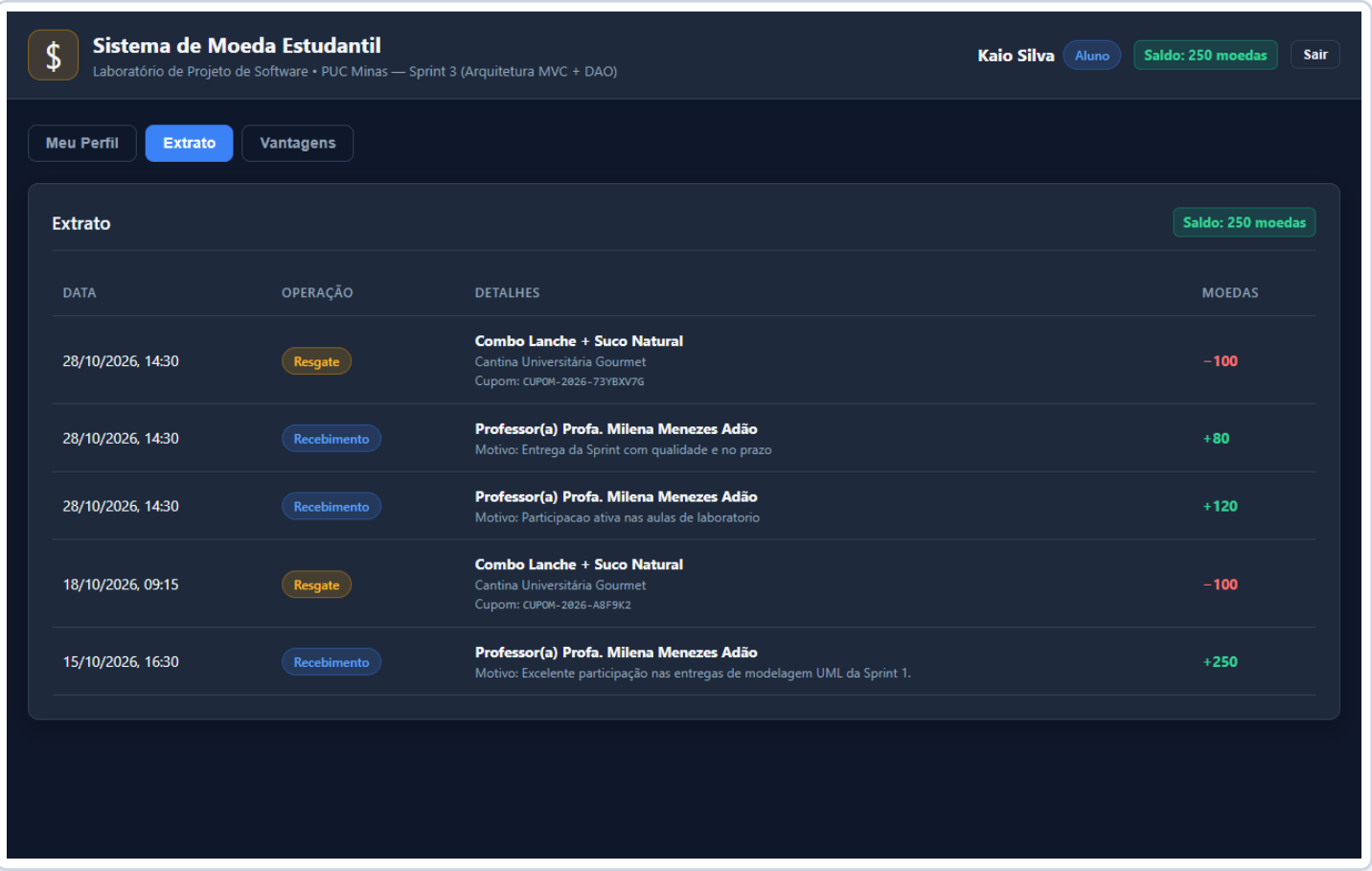




- Catálogo
- Foto
- Custo em moedas
- Empresa
- Resgatar
- Só ativas

Catálogo → GET /api/vantagens → VantagemController → VantagemService → DAO → SQLite

Tela do aluno · Extrato

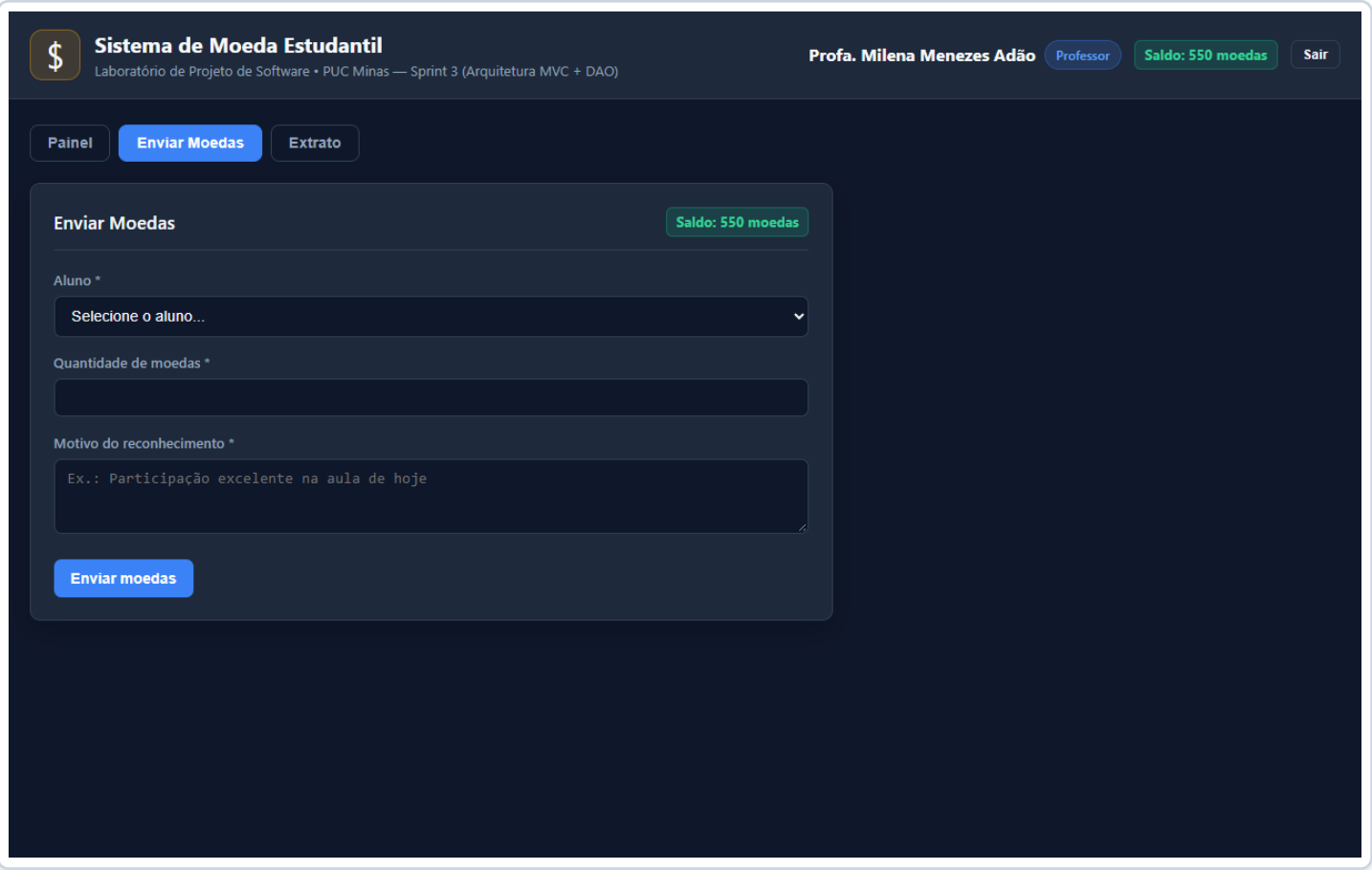




- Saldo
- Recebimentos
- Resgates
- Código do cupom

Extrato → `GET /api/extrato` → TransacaoController → TransacaoService → DAO → SQLite

Tela do professor · Enviar moedas

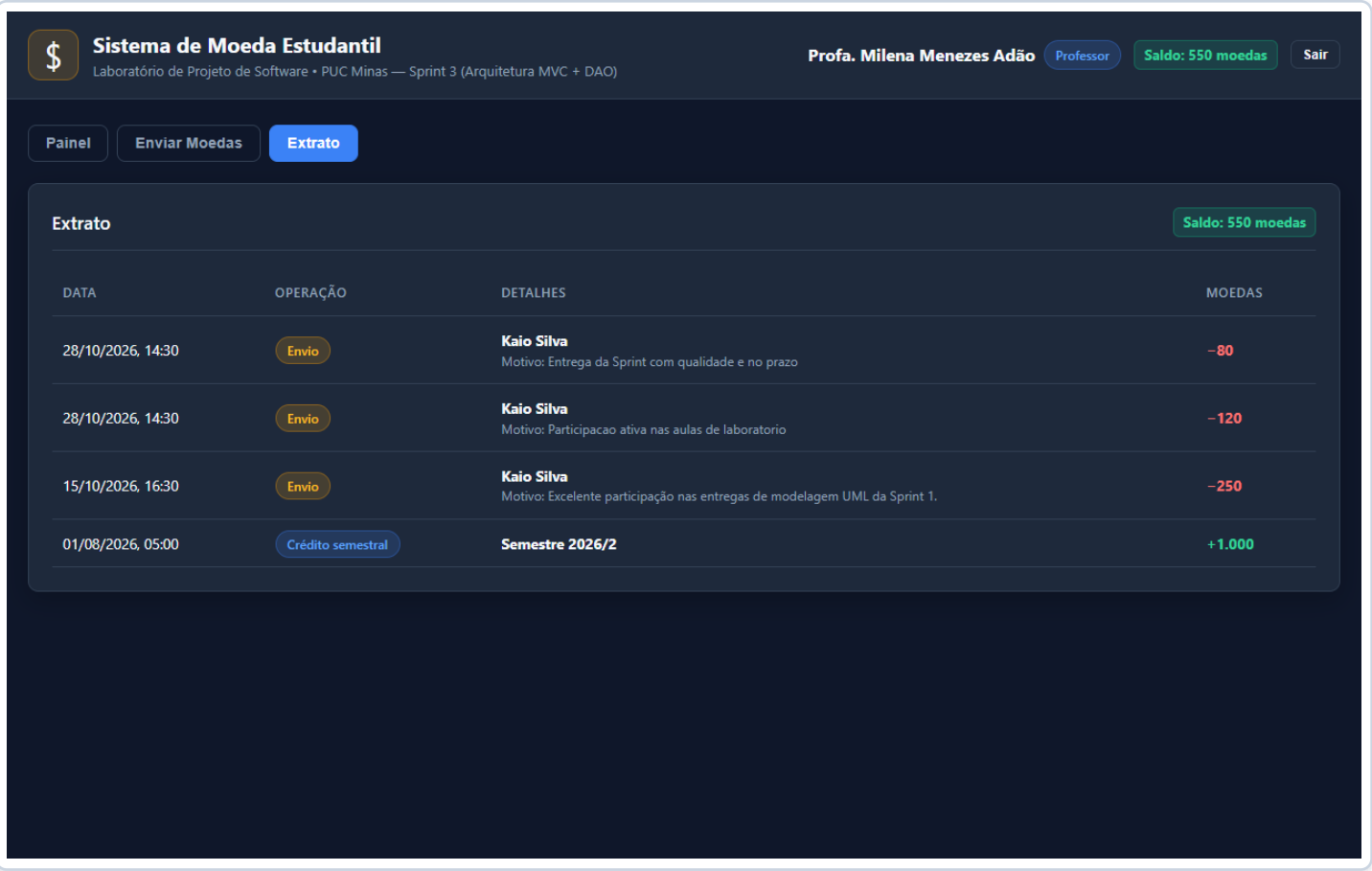




- Selecionar aluno
- Quantidade
- Motivo obrigatório
- Saldo suficiente

Formulário → `POST /api/transacoes/enviar-moedas` → TransacaoController → TransacaoService → DAO → SQLite

Tela do professor · Extrato

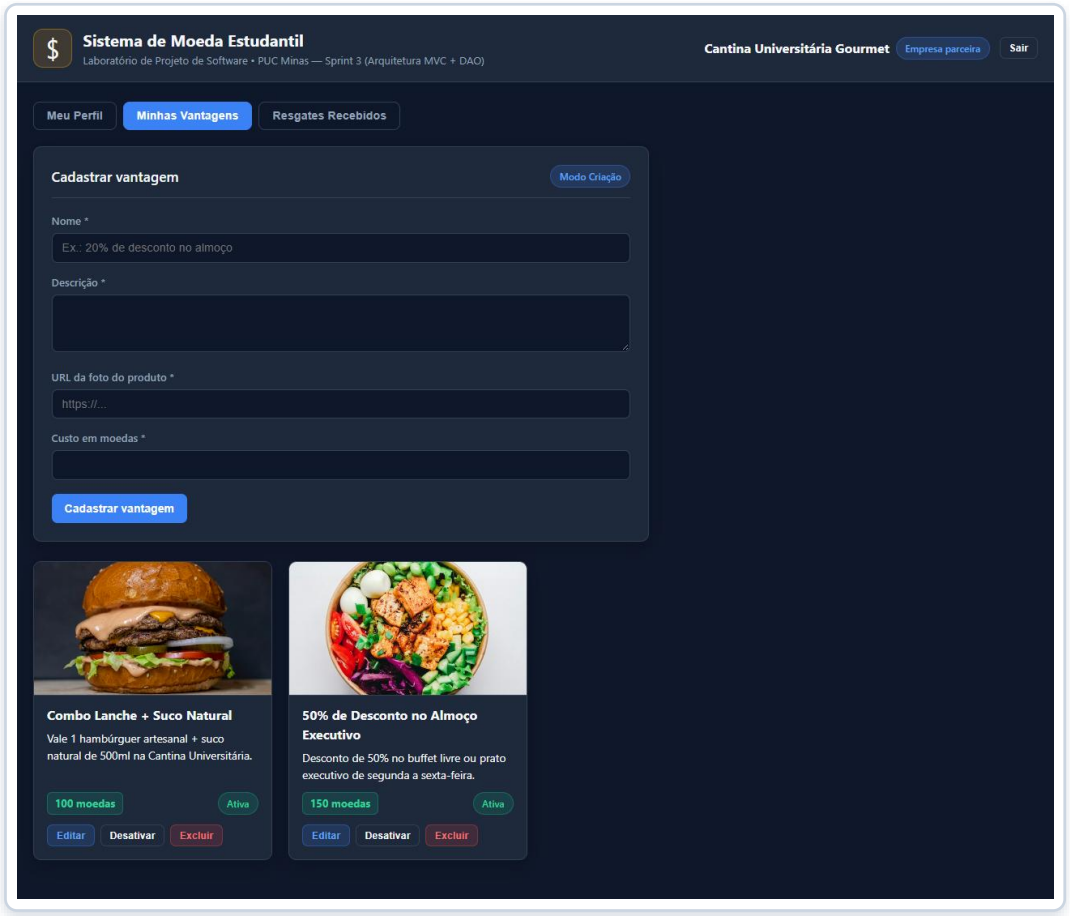




- Saldo
- Envios
- Créditos semestrais
- +1.000/semestre

Extrato → GET /api/extrato → TransacaoController → TransacaoService → DAO → SQLite

Tela da empresa · Vantagens





- Cadastrar
- Editar
- Ativar/Desativar
- Excluir
- Foto por URL
- Custo > 0

Formulário → POST /api/vantagens → VantagemController → VantagemService → DAO → SQLite

Design System e tecnologias

Design System próprio

O sistema usa tema escuro. Este slide é claro.

Fundo	Card	Primária
Sucesso	Alerta	Erro

Cards · botões · badges · tabelas · responsivo

Tecnologias



Node.js

Servidor



JavaScript

Módulos ES



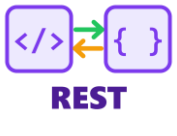
HTML/CSS

Frontend



SQLite

Banco local



API REST

HTTP padrão



node:crypto

Hash de senha



Docker

Ambiente

Testes e conclusão

Testes do DAO

```
npm run test:dao
```

`src/dao/test_dao.js`

CRUD de alunos e empresas no SQLite

Benefícios da arquitetura

- ✓ Manutenção mais simples
- ✓ Responsabilidades separadas
- ✓ Fácil de testar e evoluir



Conclusão

Sprint 3: cadastros, login, moedas, extrato, vantagens e resgate com cupom.

Perguntas?